

MAVLink explicado

Cómo se comunican el dron, la estación de tierra y los scripts en Python

Guardian Eye

TFG - Ingeniería de Tecnologías de Telecomunicación, UC3M

Nerea Gorostidi Garcia

1. Que es MAVLink y por que existe

MAVLink (Micro Air Vehicle Link) es un protocolo de mensajería binario, ligero y abierto, diseñado específicamente para que un vehículo no tripulado (un dron, un rover, un submarino) hable con una estación de tierra y con cualquier otro programa que necesite leer su telemetría o enviarle órdenes. Lo definió originalmente el mismo equipo de PX4/ArduPilot y hoy es el estándar de facto en el mundo de los drones open source.

El autopiloto (el 'cerebro' que vuela el dron: ArduPilot o PX4, corriendo dentro de una placa Pixhawk o, en un entorno de simulación, dentro de un simulador SITL) necesita comunicarse con el exterior a través de enlaces con muy poco ancho de banda: un radio-modem de telemetría a 57.6 kbps, un puerto serie, o una red WiFi/4G limitada. MAVLink resuelve esto con mensajes binarios muy compactos (unos pocos bytes de cabecera + el payload útil + un checksum), en vez de texto tipo JSON o XML, que pesaría mucho más.

1.1 Que viaja dentro de un mensaje MAVLink

- `system_id` / `component_id`: identifican quien envía el mensaje (el autopiloto suele ser `system_id=1`; una GCS o un script suele identificarse con un id distinto, p. ej. 255).
- `msg_id` / tipo de mensaje: que clase de información es (HEARTBEAT, GLOBAL_POSITION_INT, COMMAND_LONG, STATUSTEXT...). Hay varios cientos de tipos definidos.
- `payload`: los datos concretos de ese mensaje (por ejemplo, en GLOBAL_POSITION_INT van latitud, longitud, altitud y velocidades).
- `checksum`: para detectar mensajes corruptos por ruido en el enlace.

Idea clave: MAVLink en sí mismo no sabe ni le importa si viaja por un cable serie, por WiFi (UDP) o por una red con TCP. Es solo el formato de los mensajes. El 'transporte' (como llegan esos bytes de un sitio a otro) es una capa aparte, y es la que suele dar más problemas en la práctica (lo vemos en detalle en las secciones 3 y 4).

1.3 No hace falta memorizar los msg_id: hay librerías que lo evitan

Construir a mano cada mensaje MAVLink (saber que MAV_CMD_COMPONENT_ARM_DISARM es el comando correcto para armar, que parámetro va en qué posición, etc.) es tedioso y fácil de hacer mal. pymavlink -- la librería que usan los scripts de este proyecto -- ya incluye funciones de ayuda que envuelven esos mensajes con nombres claros: por ejemplo, `master.arducopter_arm()` en vez de construir un COMMAND_LONG a mano, o `master.set_mode_apm('GUIDED')` en vez de calcular el `custom_mode` numérico. Además de pymavlink, existen librerías de más alto nivel construidas encima de ella, con una API todavía más sencilla (por ejemplo, MAVSDK-Python, mantenida activamente, con llamadas como `await drone.action.arm()`). El Apéndice A, al final de este documento, recoge una tabla con los comandos MAVLink más habituales y su equivalente en pymavlink, junto con más detalle sobre estas librerías.

1.2 El mensaje mas importante: HEARTBEAT

La especificación de MAVLink espera que cada participante de la red (el autopiloto, una GCS, un companion computer/script) mande su propio mensaje HEARTBEAT aproximadamente una vez por segundo, para anunciar 'sigo aquí, este es mi tipo de sistema, este es mi modo de vuelo actual, estoy

armado o no'. Por eso el primer paso de cualquier script (`wait_heartbeat()`) es literalmente esperar a que llegue el primer HEARTBEAT del dron: si nunca llega, es la señal mas clara de que la conexión (el transporte) no está bien configurada -- el protocolo MAVLink nunca llega a intervenir.

Que la especificación 'espere' un HEARTBEAT de cada participante no significa que `arm_takeoff.py` y `telemetry.py` lo hagan: tal como están escritos ahora, los dos solo ESCUCHAN el HEARTBEAT del autopiloto (`wait_heartbeat()`) y nunca llaman a `master.mav.heartbeat_send()` para mandar el suyo propio. Funciona igual de bien porque ArduPilot, en esta configuración de SITL, no exige recibir un HEARTBEAT de vuelta para aceptar comandos ARM/TAKEOFF. Solo haría falta añadirlo si algún día activas el failsafe `FS_GCS_ENABLE` en un Pixhawk real (que sí depende de detectar la pérdida de heartbeat de la GCS) o si quieres que tu script aparezca como un participante activo mas, en vez de un oyente mudo, frente a otras herramientas MAVLink.

Que el autopiloto ya esté emitiendo su HEARTBEAT no basta para que tu script lo reciba: ese primer HEARTBEAT solo llega si Mission Planner tiene configurada, de antemano, una salida Outbound hacia el puerto exacto donde escucha el script (14551, 14552...). Sin esa salida configurada, Mission Planner nunca reenvía nada hacia ese puerto, y el script se queda esperando indefinidamente aunque el dron lleve rato mandando HEARTBEAT sin parar -- se explica el paso a paso de esta configuración en la sección 5. El síntoma es visible en el propio código de `arm_takeoff.py`: su función `connect()` informa cada pocos segundos con un mensaje del tipo 'Sin HEARTBEAT todavía (llevas Ns esperando)...' mientras no le llegue nada -- ese aviso repetido es precisamente la señal de que falta esa salida Outbound (o de que hay un interbloqueo como el descrito en la sección 4).

Figura 1. El GCS (Mission Planner o un script) y el autopiloto (ArduPilot/PX4) se hablan en ambos sentidos usando MAVLink como idioma común.

2. Los transportes de MAVLink: serie, TCP y UDP

MAVLink se puede transportar de tres formas. Cual se usa depende de si hay un cable físico directo o una red de por medio:

Transporte	Cuando se usa	Naturaleza	Ejemplo pymavlink
Serie (UART)	Conexión física directa: Pixhawk <-> Raspberry Pi, Pixhawk <-> radio telemetría	Punto a punto, un único interlocutor	'/dev/ttyAMA0', baud=921600
TCP	SITL (simulador) casi siempre lo ofrece; alguna GCS también	Orientado a conexión: hay que 'conectar' explícitamente, fiable, un flujo continuo	'tcp:127.0.0.1:5762'
UDP	Red local o Internet, telemetría por WiFi, reenvíos entre programas	Sin conexión: paquetes sueltos, no fiable por sí mismo, MAVLink añade sus propios reintentos/checks	'udp:127.0.0.1:14551'

Un entorno de SITL típico (como ArduCopter en Windows con Mission Planner) suele ofrecer varios transportes disponibles a la vez: TCP directo desde el propio simulador (puertos 5760, 5762, 5763) y, además, UDP a través de Mission Planner (14550 y los puertos extra que se configuren). Tenerlos

todos disponibles es comodo, pero es tambien la fuente de bloqueos mas habitual al conectar un script por primera vez: si no tienes claro por cual de ellos esta hablando tu script en cada momento, es facil terminar escuchando en un puerto UDP que nadie esta alimentando todavia, sin ningun mensaje de error que lo delate. La seccion 4 recorre un caso real de esto, paso a paso, para que sepas reconocerlo si te ocurre.

3. Servidor vs cliente en UDP: bind, connect, 127.0.0.1 y 0.0.0.0

Esta es la parte que mas bloqueos suele causar en la practica, y vale la pena entenderla a fondo porque se repite en cualquier programa que hable por red.

3.1 UDP no tiene 'conexion' real

A diferencia de TCP, UDP no establece una conexion formal entre dos extremos. Cada paquete UDP viaja de forma independiente, con su IP:puerto de origen y de destino, y punto. El 'cliente' o 'servidor' no es un concepto del protocolo UDP en si -- es una convencion que depende de que llamada de la libreria de sockets se use primero:

- `bind(direccion, puerto)`: reserva ese puerto localmente y deja el proceso escuchando de forma pasiva. No manda nada por iniciativa propia. Solo cuando le llega el primer paquete, aprende la IP:puerto de quien se lo mando y, si quiere, le puede responder a esa direccion.
- `sendto(direccion, puerto)`: manda activamente un paquete a una direccion conocida de antemano. No necesita reservar un puerto propio fijo (el sistema operativo le asigna uno libre cualquiera), solo necesita saber a donde disparar.

En pymavlink esto se traduce directamente en la cadena de conexion:

- `'udp:HOST:PUERTO'` y `'udp:HOST:PUERTO'` (son sinonimos) -> hacen `bind()` -> modo SERVIDOR.
- `'udpout:HOST:PUERTO'` -> hace `sendto()` -> modo CLIENTE, tiene que hablar primero.

El detalle importante -- y la fuente de confusion mas habitual -- es que la forma corta `'udp:'` NO es neutra: pymavlink la trata igual que `'udp:'`, es decir, en modo SERVIDOR por defecto.

Figura 2. Funcionamiento correcto: uno de los dos procesos hace `bind` y espera (servidor), el otro manda el primer paquete (cliente). A partir de ahi, el servidor ya conoce la direccion del cliente y puede responderle.

3.2 127.0.0.1 frente a 0.0.0.0: no son 'el mismo puerto'

Cuando haces `bind()`, tambien indicas en que direccion IP local vas a escuchar. Aqui es facil confundirse porque los dos valores mas comunes parecen intercambiables y no lo son:

- 127.0.0.1 (localhost / loopback): solo acepta paquetes que se originan en la misma maquina. Es una red virtual interna que ni siquiera sale por la tarjeta de red. Ideal para procesos que conviven en el mismo PC (por ejemplo, SITL, Mission Planner y un script, todos en el mismo Windows).
- 0.0.0.0 (todas las interfaces): significa 'acepta paquetes que lleguen por cualquier interfaz de red de esta maquina' -- la de loopback, la WiFi, la Ethernet, una VPN, etc. Se usa cuando otro equipo de la red (por ejemplo, una tablet con Mission Planner, o la propia Raspberry Pi) necesita conectarse desde fuera.

Esta diferencia puede producir una situacion confusa: dos procesos distintos pueden reservar el mismo numero de puerto sin que el sistema operativo se queje, siempre que uno escuche en la

direccion generica (0.0.0.0) y el otro en una direccion mas especifica (127.0.0.1) -- no son exactamente el mismo recurso a nivel de sistema operativo, asi que ninguno de los dos falla al reservarlo. La causa tipica es mezclar, sin darse cuenta, dos programas que abren 'el mismo puerto' pero en direcciones distintas: cada uno se queda escuchando tranquilamente en su propio rincon, y ninguno ve nunca los paquetes del otro, sin que salte ningun error de 'puerto ocupado' que lo delate. Para evitarlo: cuando dos procesos deban hablar por el mismo puerto, usa siempre la misma direccion de bind en ambos lados (lo mas simple, en un mismo PC, es que los dos usen 127.0.0.1) y comprueba con netstat/ss, ante cualquier duda, en que direccion exacta esta escuchando cada uno.

4. Como reconocer y resolver un interbloqueo tipico en UDP

4.1 El sintoma: el script se queda esperando el HEARTBEAT para siempre

Cuando un script abre una conexion con 'udp:HOST:PUERTO' (modo servidor) y el proceso que deberia alimentar ese puerto -- por ejemplo Mission Planner -- tambien esta escuchando pasivamente en ese mismo puerto, sin tener configurada ninguna salida activa hacia el, el resultado es que ninguno de los dos manda nunca el primer paquete. El script se queda colgado en wait_heartbeat() de forma indefinida, sin ningun mensaje de error: el bind() en si no falla, asi que no hay nada que avise de que algo va mal.

Figura 3. Interbloqueo tipico: dos procesos en modo servidor sobre el mismo puerto, ninguno manda el primer paquete.

4.2 Como diagnosticarlo

- Revisa los puertos con netstat (Windows) o ss/netstat (Linux): si aparece otro proceso ya escuchando en modo servidor sobre ese mismo puerto (sea en 0.0.0.0 o en 127.0.0.1), es la senal mas clara de este interbloqueo.
- Forzar el script a modo cliente no basta por si solo: cambiar la cadena a 'udpout:HOST:PUERTO' y mandar un HEARTBEAT propio antes de escuchar, para 'darse a conocer', solo funciona si el otro extremo esta realmente configurado para reenviar datos a quien le escriba. Si ese puerto no tiene ninguna salida activa configurada (por ejemplo, esta reservado para otra funcion interna del programa), seguira sin llegar nada: el bind() por si solo nunca garantiza reenvio, eso es logica de la aplicacion, no algo que UDP resuelva automaticamente.

4.3 Como evitarlo

La forma correcta de resolverlo es configurar, en el lado que hace de Bridge (Mission Planner u otro), una salida activa (Outbound) hacia la direccion y puerto exactos donde tu script esta escuchando -- se explica paso a paso en la seccion 5. Como alternativa rapida para descartar por completo el problema mientras diagnosticas, tambien puedes conectar el script directamente a un puerto TCP libre que ofrezca el propio SITL, sin depender de ningun reenvio intermedio:

Figura 4. Dos caminos posibles desde el SITL: uno via Mission Planner sin ninguna salida configurada (bloqueado), y otro directo a un puerto TCP libre del SITL (funciona sin depender de ningun reenvio).

Conectar directamente a un puerto TCP libre del SITL es util para descartar rapido si el problema esta en el reenvio de Mission Planner, pero mas abajo (seccion 6) se explica como configurar correctamente ese reenvio si prefieres mantener Mission Planner en medio -- por ejemplo, para seguir

viendo el vuelo en su interfaz a la vez que tu script actúa.

5. Que funcion cumple Mission Planner en todo esto

Mission Planner es una GCS (estacion de tierra): un programa pensado para que una persona vea el mapa, la telemetria, configure parametros del autopiloto y pueda planificar/enviar misiones. Cuando lanzas la simulacion desde Mission Planner (boton 'Simulate'), el propio Mission Planner:

- Arranca el proceso ArduCopter.exe (el SITL) en segundo plano.
- Se conecta a el como cliente MAVLink por TCP (puerto 5760).
- Opcionalmente, puede reenviar ('forward') ese mismo flujo de telemetria a otros programas, por UDP, para que herramientas externas (otra GCS, un script) tambien puedan verlo sin competir por el puerto 5760.

Ese reenvio NO esta activado por defecto en general -- hay que configurarlo explicitamente en Mission Planner, indicando a que IP:puerto quieres que Mission Planner envíe una copia de los datos. Si dejas un puerto sin configurar (por ejemplo, si tu script escucha en el 14551 pero nunca anades esa salida), el puerto puede aparecer abierto en el sistema pero no estar realmente sirviendo telemetria a nadie.

5.1 Como configurar el reenvio de puertos en Mission Planner

- Con Mission Planner conectado al dron (real o SITL, viendo ya datos de vuelo), pulsa Ctrl+F para abrir el panel de opciones avanzadas ('temporary flight data' / acciones ocultas).
- Pulsa el boton Mavlink. Se abre una ventana llamada 'SerialOutput - Mavlink' con una tabla donde cada fila es una salida de reenvio.
- Rellena una fila con los datos de la salida que quieres crear (ver tabla de columnas mas abajo) y, muy importante, pulsa el boton Go de esa misma fila -- sin eso, la fila queda escrita pero la salida NO se activa. Es un fallo muy habitual: rellenar el puerto pero dejar el campo Host vacio, o rellenarlo todo y olvidar pulsar Go, hace que la fila parezca configurada sin estarlo realmente, y el script se queda esperando sin recibir nada.

5.2 Las columnas de la ventana SerialOutput - Mavlink

Columna	Que significa
Type	El transporte de esa salida: UDP, TCP o Serial. Para reenviar a un script en el mismo PC, siempre UDP.
Direction	Outbound o Inbound (ver regla completa en 5.3). Es la decision mas importante de la fila.
Port	El numero de puerto de esa salida (por ejemplo, 14551 o 14552, uno por cada programa que quieras alimentar).
Host/Baud	La IP de destino cuando Direction=Outbound (sin esto, Mission Planner no sabe a quien enviar). Se usa para Baudrate solo si Type=Serial.
Write	Casilla que activa realmente el envio de datos por esa fila (ademas de pulsar Go).
Go	Boton que aplica y arranca la fila. Hay que pulsarlo siempre tras rellenar los demas campos.

5.3 Outbound vs Inbound: la regla que decide si tu script es servidor o cliente

Esta columna Direction es, en el fondo, la misma decision de bind() vs sendto() que vimos en la seccion 3, pero vista desde el lado de Mission Planner en vez del lado de tu script:

Direction en Mission Planner	Que hace Mission Planner	Que debe hacer tu script en Python
Outbound	Envia activamente (sendto) el flujo MAVLink hacia el Host:Port que le indiques. Mission Planner se comporta como CLIENTE de esa salida.	SERVIDOR: abrir el puerto en modo bind y esperar ('udp:127.0.0.1:PUERTO' o 'udp:127.0.0.1:PUERTO'). Es lo que usan arm_takeoff.py y telemetry.py.
Inbound	Mission Planner se queda escuchando (bind) pasivamente ese puerto, esperando que algo LE ENVIE datos a ella. Se comporta como SERVIDOR.	CLIENTE: tu script debe mandar activamente hacia Mission Planner ('udpout:127.0.0.1:PUERTO'). Se usaria al revés del caso habitual: por ejemplo, si un script generase telemetria extra y quisieras inyectarsela a Mission Planner.

Regla practica para decidir en 5 segundos: si en Mission Planner pones Outbound, en tu script usas 'udp:' (servidor). Si algun dia pones Inbound, en tu script usas 'udpout:' (cliente). Para que arm_takeoff.py y telemetry.py RECIBAN una copia de la telemetria, siempre hace falta Outbound en Mission Planner + servidor en el script -- es la combinacion que usan las dos filas de la seccion 6.

5.4 El GCS es bidireccional: dos caminos posibles para los comandos

Es facil pensar en Mission Planner como un simple 'visor' de telemetria, pero no lo es: un GCS es, en si mismo, un participante MAVLink completo y bidireccional. Cada vez que un piloto pulsa un boton en su interfaz -- armar, cambiar de modo, RTL -- Mission Planner esta generando y enviando un comando MAVLink hacia el vehiculo, exactamente igual que lo hace arm_takeoff.py por codigo. La telemetria (dron -> GCS) es solo la mitad 'de subida' de ese enlace; la mitad 'de bajada' (comandos GCS -> dron) existe siempre, la use un humano o un script.

Esto tiene una consecuencia directa sobre por donde viajan los comandos de un script: hay dos caminos posibles, y ambos son validos.

- Camino A -- directo al SITL: si el script se conecta a un puerto TCP libre del simulador (seccion 4.3, p.ej. 'tcp:127.0.0.1:5762'), sus comandos van directos al vehiculo, sin pasar por Mission Planner en ningun momento. Si Mission Planner esta conectado a la vez (por su propio TCP 5760), vera los efectos en la telemetria, pero no participa en el envio del comando.
- Camino B -- a traves de Mission Planner (Bridge): si el script se conecta al puerto UDP que Mission Planner tiene configurado como Outbound (seccion 5.1), ese enlace es bidireccional. Mission Planner recibe de vuelta, por el mismo socket, los comandos que manda el script, y los reenvia hacia el SITL a traves de su propia conexion TCP interna (5760) -- el mismo papel de repetidor que cumple con cualquier otro comando, sea de un script o generado por el propio piloto desde la interfaz.

Figura 5. Los dos caminos posibles para los comandos de un script. En el Camino B, Mission Planner

reenvia tanto telemetria (subida) como comandos (bajada) -- el enlace Outbound nunca es de un solo sentido.

En el proyecto tal como esta configurado (seccion 6), arm_takeoff.py y telemetry.py usan el Camino B: por eso el ARM y el TAKEOFF que manda arm_takeoff.py llegan al SITL aunque el script nunca se conecte directamente a el -- viajan a traves de la misma salida Outbound por la que llega la telemetria, en sentido contrario.

6. Los dos programas reales: arm_takeoff.py (comandos) y telemetry.py (telemetria)

Es una buena practica separar responsabilidades: un proceso que decide y manda ordenes (armar, cambiar de modo, despegar, ir a un punto) y otro proceso, independiente, que solo lee telemetria y la muestra o la registra (por ejemplo, un dashboard). MAVLink lo permite de forma natural, porque cada mensaje ya lleva su propio tipo: cualquier receptor puede quedarse solo con los mensajes que le interesan e ignorar el resto. Esto es exactamente lo que tenemos ahora en el proyecto: dos scripts independientes, cada uno con su propio puerto.

6.1 La configuracion recomendada

Por que servidor y no cliente: las dos filas de Mission Planner estan configuradas como Outbound, con un Host y un Port ya escritos a mano -- Mission Planner conoce de antemano la direccion exacta y dispara los datos hacia alli sin esperar a que nadie se los pida (es el CLIENTE de esa relacion). Para que esos paquetes lleguen a algun sitio, tiene que haber alguien ya esperando en esa direccion y puerto exactos, con el puerto reservado (bind()), antes de que Mission Planner mande el primer paquete -- ese papel de 'alguien ya esperando' es el SERVIDOR, y es el que cumplen arm_takeoff.py y telemetry.py. Si los scripts tambien intentaran ser clientes ('udpout:'), no habria nadie escuchando en el 14551/14552 cuando Mission Planner mandara sus paquetes, y el sistema operativo los descartaria sin que nadie los recogiera. No es una cuestion de quien 'quiere' los datos, sino de quien se compromete primero con una direccion fija (ver la regla completa en 5.3).

- arm_takeoff.py: arma el dron y manda el despegue. Escucha en modo SERVIDOR ('udp:127.0.0.1:14551').
- telemetry.py: solo lee posicion/altura y las muestra por pantalla. Escucha en modo SERVIDOR ('udp:127.0.0.1:14552').
- Mission Planner: tiene dos filas Outbound configuradas en su ventana SerialOutput - Mavlink (seccion 5.2): una hacia 127.0.0.1:14551 y otra hacia 127.0.0.1:14552, ambas con Write activado y el boton Go pulsado.

```
arm_takeoff.py -> mavutil.mavlink_connection('udp:127.0.0.1:14551') # servidor
```

```
telemetry.py -> mavutil.mavlink_connection('udp:127.0.0.1:14552') # servidor
```

El motivo de usar dos puertos distintos NO es que MAVLink necesite separar comandos de telemetria (todo viaja mezclado por el mismo cable/red y cada script filtra lo que le interesa por tipo de mensaje) -- es que dos procesos de sistema operativo distintos no pueden compartir de forma fiable el mismo puerto UDP en modo servidor. Si arm_takeoff.py y telemetry.py intentaran hacer bind() los dos sobre el 14551 a la vez, aparece el mismo tipo de conflicto descrito en la seccion 4: dos procesos reservando el mismo recurso, sin garantia de quien recibe cada paquete.

Figura 6. Arquitectura real en marcha: Mission Planner actua de MAVLink Bridge, recibe un unico flujo de SITL por TCP y lo redistribuye (Outbound) hacia dos salidas UDP independientes, una por script. Importante: si mandas comandos (armar, despegar...) desde arm_takeoff.py, ese trafico tiene que volver hacia el autopiloto pasando otra vez por Mission Planner. Las salidas Outbound de Mission Planner son bidireccionales -- por eso arm_takeoff.py puede tanto recibir el HEARTBEAT como mandar los comandos ARM/TAKEOFF por el mismo socket. Si algun dia una salida no te devuelve respuesta a tus comandos (solo telemetria), puedes recurrir a la alternativa de la seccion 4.3: conectar ese script directamente a un puerto TCP libre de SITL.

7. ¿Quien es responsable de mandar el primer HEARTBEAT?

Esta pregunta mezcla dos capas distintas que conviene separar con claridad: el protocolo MAVLink en si, y la fontaneria del transporte de red (UDP), que es donde suelen aparecer los problemas descritos en las secciones anteriores.

7.1 A nivel de MAVLink: siempre es el autopiloto (y la GCS, en sentido contrario)

El HEARTBEAT no es una respuesta a nada -- no forma parte de un 'saludo' pregunta/respuesta. Es una emision periodica, unilateral y continua: el autopiloto (ArduPilot en el SITL, o un Pixhawk real) empieza a mandar su HEARTBEAT en el instante en que arranca, a razon de aproximadamente 1 vez por segundo, y sigue haciendolo para siempre, exista o no exista alguien escuchando al otro lado. Tu script nunca tiene que 'despertar' al dron ni mandarle nada primero para que el autopiloto empiece a hablar. En sentido contrario, una GCS o companion computer 'correcto' tambien deberia emitir su propio HEARTBEAT hacia el autopiloto -- algunos failsafes de ArduPilot usan la ausencia de HEARTBEAT de la GCS para decidir que se ha perdido el enlace -- pero eso es aparte de simplemente poder OIR al dron, que es lo unico necesario para que un script funcione.

7.2 A nivel de transporte UDP: no es un problema de MAVLink en si

El interbloqueo descrito en la seccion 4 no tiene nada que ver con quien 'debe' hablar primero segun el protocolo -- es una limitacion pura de como funciona un socket UDP en modo servidor: por mucho que el autopiloto este ya emitiendo HEARTBEAT sin parar, un proceso que solo ha hecho bind() no tiene forma de recibir esos datos si nadie le ha mandado nunca ni un solo paquete a el, porque el sistema operativo (y, si hay un Bridge como Mission Planner en medio, tambien el Bridge) no sabe todavia a que direccion IP:puerto reenviar la copia. Es un problema de enrutado/plumbing, resuelto configurando la salida Outbound -- no es que el autopiloto se niegue a hablar primero.

Figura 7. El autopiloto siempre emite HEARTBEAT por iniciativa propia. Lo que cambia entre el entorno SITL+Mission Planner (UDP) y un Pixhawk real por serie es solo la fontaneria necesaria para que ese HEARTBEAT llegue hasta tu script.

7.3 ¿Cambia esto con un Pixhawk real conectado por serie?

No, la responsabilidad del HEARTBEAT sigue siendo del autopiloto -- eso no cambia nunca, sea SITL o hardware real. Lo que SI desaparece por completo es el problema de enrutado UDP: un puerto serie es un cable fisico directo, sin direcciones IP, sin bind(), sin Inbound/Outbound y sin ninguna ambigüedad de quien es cliente o servidor. En cuanto el Pixhawk arranca, sus HEARTBEAT viajan literalmente por el cable TX/RX hacia la Raspberry; tu script solo tiene que abrir ese dispositivo serie y

leer -- no hay nada que 'conectar' ni configurar para que el primer paquete llegue, porque en un cable punto a punto no hace falta que nadie aprenda la direccion de nadie.

En resumen: el HEARTBEAT siempre lo manda el autopiloto por iniciativa propia, tanto en SITL como en un Pixhawk real. Todo lo que hay que vigilar (bind, Outbound/Inbound, quien manda el primer paquete UDP) es exclusivamente fontaneria de la red, que desaparece en cuanto la conexion pasa a ser un cable serie directo.

8. El paso siguiente: Raspberry Pi conectada directamente al Pixhawk

Todo lo anterior (SITL, TCP, UDP, Mission Planner reenviando puertos) es la situacion de desarrollo/simulacion en tu PC. Cuando la Raspberry Pi este conectada fisicamente al Pixhawk del dron real (normalmente por un cable a un puerto TELEM libre del Pixhawk, o directamente a los pines UART de la Raspberry), el panorama cambia de raiz:

Los puertos TCP libres del SITL (5762, 5763) que usamos en la seccion 4.3 no son un capricho de la simulacion: existen precisamente para imitar, en software, algo que en el hardware real ya viene resuelto de fabrica. Una Pixhawk fisica trae varios puertos serie (UART) independientes -- normalmente etiquetados TELEM1, TELEM2, etc. -- cada uno de ellos ya es, por si solo, un canal exclusivo con su propio cable. Por eso en hardware real no hace falta ningun Bridge ni ningun reenvio de puertos: cada consumidor (la GCS de campo, la Raspberry Pi) se conecta a su propio puerto TELEM fisico, sin compartir nada con nadie.

Figura 8. Los puertos TCP simulados del SITL (5760, 5762, 5763) imitan en software lo que en la Pixhawk real ya son puertos serie fisicos independientes (TELEM1, TELEM2...).

- Ya no hay red IP entre la Raspberry y el Pixhawk. Es un cable serie (UART) fisico, con lineas TX, RX y GND. No existe el concepto de 'puerto de red', ni bind, ni 127.0.0.1, ni 0.0.0.0 -- todo eso es propio de las redes IP, no del puerto serie.
- No hay 'cliente' ni 'servidor'. Un puerto serie es un enlace punto a punto full-duplex exclusivo: solo hay dos extremos posibles (el Pixhawk y la Raspberry) y ambos pueden hablar a la vez, no hace falta que nadie 'escuche primero'.
- Solo un proceso puede abrir el dispositivo serie a la vez. A diferencia de SITL, que podia ofrecer 3 puertos TCP simultaneos para 3 clientes distintos, un puerto serie fisico (/dev/ttyAMA0, /dev/serial0...) normalmente solo lo puede tener abierto un proceso al mismo tiempo.
- Hay que igualar la velocidad (baudrate). El parametro SERIALx_BAUD configurado en el Pixhawk (para el puerto TELEM que uses) debe coincidir exactamente con el baudrate que le indiques a pymavlink, o la comunicacion no se entendera aunque el cable este bien conectado.

```
mavutil.mavlink_connection('/dev/serial0', baud=921600) # en la Raspberry (Linux)
```

```
mavutil.mavlink_connection('COM3', baud=921600) # si pruebas por USB en Windows
```

8.1 Como tener, aun asi, varios programas (comandos + telemetria) en la Raspberry

Como el puerto serie solo admite un dueño, si quieres mantener la misma separacion en dos procesos (uno de comandos, otro de telemetria) necesitas un 'concentrador' intermedio que sea el unico que abre el puerto serie, y que reparta esos datos hacia dentro de la propia Raspberry por red

local (localhost) a varios procesos -- exactamente el mismo patron que hacia Mission Planner con SITL, solo que ahora la fuente es el cable serie real en vez de un TCP de simulacion. Las herramientas tipicas para este papel son mavlink-router o mavproxy, ambas pensadas para correr en un companion computer como la Raspberry.

Figura 9. Con el dron real: un unico enlace serie exclusivo Pixhawk-Raspberry, repartido internamente por mavlink-router/mavproxy hacia varios procesos Python locales, igual que Mission Planner hacia con SITL.

En resumen: cuando migres de SITL a la Raspberry+Pixhawk real, cambia unicamente la cadena de conexion (de 'tcp:...' o 'udp:...' a la ruta del dispositivo serie con su baudrate) en el proceso que habla directamente con el Pixhawk. Si mantienes varios scripts, deja que solo uno de ellos (o una herramienta como mavlink-router) sea el que abre el puerto serie, y que los demas sigan hablando entre si por UDP local como ya hacen ahora -- esa parte del diseno no cambia.

Este paso -- Raspberry Pi controlando por TELEM2, GCS de campo recibiendo telemetria por radio en TELEM1 -- corresponde a la 'Fase 5: Despliegue en hardware real' del documento Arquitectura_Teleoperacion_Simulacion_Drones.docx, que sitúa este momento dentro del recorrido completo de ocho etapas del proyecto (desde el estudio del arte hasta el panel de control en la nube). Consulta ese documento para ver como encaja este paso con lo que viene antes y despues.

9. Resumen rapido / chuleta

Concepto	Significa
MAVLink	Formato binario de mensajes; no depende del medio de transporte.
HEARTBEAT	Mensaje de 'sigo vivo' que manda el autopiloto por iniciativa propia ~1 vez por segundo, desde que arranca. Nunca lo tiene que provocar el script.
bind()	Reservar un puerto local y quedarse escuchando pasivamente (servidor).
connect()/sendto()	Mandar activamente el primer paquete a un destino conocido (cliente).
udp: / udpin:	En pymavlink, ambas abren el socket en modo servidor (bind).
udpout:	En pymavlink, abre el socket en modo cliente (envia primero).
Outbound (Mission Planner)	MP envia activamente (cliente) a un Host:Port -> tu script debe ser servidor ('udp:').
Inbound (Mission Planner)	MP escucha pasivamente (servidor) -> tu script debe ser cliente ('udpout:').
127.0.0.1	Loopback: solo trafico que se origina en la misma maquina.
0.0.0.0	Todas las interfaces de red de la maquina: acepta trafico desde fuera tambien.
SITL	Software In The Loop: ArduPilot/PX4 corriendo como simulador en tu PC.

Concepto	Significa
Puerto serie (UART)	Enlace fisico punto a punto; sin IP, sin bind, sin cliente/servidor, un solo dueño posible.
mavlink-router / mavproxy	Reparte un unico enlace (serie o de red) hacia varios programas a la vez.

Apendice A -- Comandos MAVLink habituales, modo GUIDED y misiones

A.1 Requisito previo: el vehiculo debe estar en modo GUIDED

ArduCopter solo acepta la mayoría de comandos de vuelo dirigidos por un ordenador externo -- armar y despegar mediante MAV_CMD_NAV_TAKEOFF, moverse a un punto concreto, etc. -- cuando esta en un modo de vuelo que delega el control en quien se lo pide. En ArduCopter, ese modo es GUIDED. Si el vehiculo esta en un modo manual (STABILIZE, LOITER, ALT_HOLD...), estos comandos autonomos se rechazan aunque el vehiculo este correctamente armado.

Hay dos formas de poner el vehiculo en GUIDED:

- Manualmente, en Mission Planner: en la pestaña Actions, o con el selector de modo de vuelo (Flight Mode) del panel principal, elige GUIDED del desplegable antes de mandar ningun comando desde un script.
- Por script, como primer paso: que el propio programa mande el modo GUIDED antes de armar. Es exactamente lo que hace arm_takeoff.py en este repositorio: llama a set_mode(master, "GUIDED") antes de arm(master), precisamente por este motivo.

Figura 10. Secuencia obligatoria: primero GUIDED (manual o por script), despues ARM y TAKEOFF -- estos dos ultimos solo se aceptan una vez en GUIDED.

A.2 Tabla de comandos MAVLink habituales

Accion	Como se pide en MAVLink	Ejemplo en pymavlink
Cambiar a modo GUIDED	SET_MODE (custom_mode del modo GUIDED)	master.set_mode_apm('GUIDED')
Armar motores	MAV_CMD_COMPONENT_ARM_DISARM (param1=1)	master.arducopter_arm()
Desarmar	MAV_CMD_COMPONENT_ARM_DISARM (param1=0)	master.arducopter_disarm()
Despegar	MAV_CMD_NAV_TAKEOFF (param7=altitud objetivo)	command_long_send(..., MAV_CMD_NAV_TAKEOFF, 0, 0,0,0,0,0, alt)
Volver al punto de despegue (RTL)	Cambiar el modo de vuelo a RTL	master.set_mode_apm('RTL')
Aterrizar en el sitio	Cambiar el modo de vuelo a LAND	master.set_mode_apm('LAND')
Ir a un punto en tiempo real (GUIDED)	SET_POSITION_TARGET_GLOBAL_INT	master.mav.set_position_target_global_int_send(...)
Subir una mision (varios puntos)	Protocolo de misiones: MISSION_COUNT + MISSION_ITEM_INT (uno por punto)	master.mav.mission_count_send(...) / mission_item_int_send(...)

Accion	Como se pide en MAVLink	Ejemplo en pymavlink
Iniciar la mision ya subida	Cambiar el modo de vuelo a AUTO	master.set_mode_apm('AUTO')

Estos nombres (`master.arducopter_arm()`, `set_mode_apm(...)`...) son metodos de ayuda que ya trae pymavlink: por dentro arman el mensaje MAVLink correcto (`COMMAND_LONG`, `SET_MODE...`) sin que tengas que conocer sus identificadores numericos ni el orden exacto de los parametros. Ver A.4 para mas librerias con este mismo objetivo.

A.3 El concepto de mision y waypoints

Un waypoint es un punto geografico (latitud, longitud, altitud) al que se asocia un comando -- normalmente 'navega hasta aqui' (`MAV_CMD_NAV_WAYPOINT`), pero tambien puede ser 'despega' (`MAV_CMD_NAV_TAKEOFF`), 'atteriza' (`MAV_CMD_NAV_LAND`), 'espera N segundos' (`MAV_CMD_CONDITION_DELAY`) o 'salta a otro punto de la lista' (`MAV_CMD_DO_JUMP`), entre otros.

Una mision es una lista ordenada de waypoints que se sube una sola vez al autopiloto (mediante el protocolo de misiones de MAVLink: el GCS o el script anuncia cuantos puntos va a mandar con `MISSION_COUNT`, el autopiloto los va pidiendo uno a uno con `MISSION_REQUEST`, y cada uno se envia con `MISSION_ITEM_INT`, terminando con un `MISSION_ACK` de confirmacion). Una vez subida, la mision queda almacenada a bordo del autopiloto, listo para ejecutarla de forma autonoma.

La diferencia clave frente al modo `GUIDED` es esta: en `GUIDED`, cada movimiento lo decide en tiempo real quien tiene el control (un script o la GCS), comando a comando. En modo `AUTO`, el vehiculo ya tiene la mision completa cargada de antemano y la ejecuta por su cuenta, punto tras punto, sin necesidad de que nadie le siga mandando ordenes en cada instante -- incluso puede seguir volando la mision si se corta temporalmente el enlace con la GCS, segun la configuracion de failsafe.

Mission Planner incluye un editor visual de misiones (pestaña Plan) que genera y sube esta misma secuencia de mensajes sin necesidad de escribir codigo: se colocan los puntos sobre el mapa y el propio programa construye los `MISSION_ITEM_INT` correspondientes.

A.4 Librerias que evitan trabajar con los msg_id a mano

Ademas de los metodos de ayuda que ya incluye pymavlink (tabla de A.2), existen librerias de mas alto nivel construidas encima de MAVLink, pensadas para no tener que pensar en mensajes ni en identificadores en absoluto:

- pymavlink (la que usa este proyecto): nivel intermedio. Sigue siendo MAVLink de cerca (cada mensaje, cada campo), pero con funciones de ayuda con nombre (`arducopter_arm()`, `set_mode_apm(...)`) que evitan calcular a mano los identificadores numericos de comandos y modos.
- DroneKit-Python: libreria historica, muy usada durante años con ArduPilot, con una API orientada a objetos (`vehicle.armed = True`, `vehicle.simple_takeoff(alt)`). Ya no tiene mantenimiento activo, por lo que conviene tenerlo en cuenta antes de empezar un proyecto nuevo con ella.
- MAVSDK-Python: la alternativa moderna y mantenida activamente (proyecto Dronecode), con una API asincrona (`await drone.action.arm()`, `await drone.action.takeoff()`) que funciona tanto con PX4 como con ArduPilot.

Recomendacion practica: para scripts sencillos como los de este repositorio, los metodos de ayuda de pymavlink (tabla A.2) suelen ser suficientes. Para un proyecto con logica de mision mas compleja,

una libreria de mas alto nivel como MAVSDK-Python puede ahorrar bastante codigo repetitivo a cambio de perder algo de control fino sobre los mensajes.